

ArcSettle: Reserve-Gated USDC Settlement with On-Chain Proof-of-Reserves on Arc

Dipankar Sarkar
doom2quake

August 2026

Abstract

Fiat-backed stablecoins settle on-chain but prove solvency off-chain. The chain sees tokens move; it does not see whether those tokens are backed at the instant they are settled. That gap between reported reserves and reserves proven *right now* is where depegs and insolvency events live, and it sharpens the moment an autonomous treasury agent settles or mints on a schedule against a figure it last read. We present ArcSettle, a settlement rail for Arc, Circle’s settlement chain, whose every settlement is gated on-chain by a signed proof-of-reserves attestation: settled supply provably cannot exceed the USDC reserves that have actually been proven. A redemption does *not* reopen settlement headroom until a new attestation has observed the outflow, which closes a redeem-then-resettle loop that naive reserve gates admit. A reserve-aware operator agent runs the same gate off-chain first, so a doomed settlement never spends gas. This paper describes the design and the milestone-1 implementation: two self-contained Solidity contracts and an operator agent, verified by 59 Solidity tests and 75 Python tests. All work is Arc testnet only; no mainnet, no live broadcast, no users, and no audit (Section 5).

1 Problem

A regulated dollar stablecoin such as USDC [5] holds reserves at a bank and asserts that circulating supply never exceeds them. The assertion is an off-chain attestation; the chain enforces nothing about it. Three failure shapes follow. First, an attestation *lags*: supply is settled against a reserve figure that a bank outflow has already invalidated. Second, a *redemption* frees mint or settlement headroom the instant supply falls, before the corresponding reserve outflow is observed, so an operator can redeem, move the freed collateral out, and re-settle against the same stale proof. Third, a *stale or future-dated* timestamp is treated as eternally fresh. Anyone who accepts the token as settlement — a merchant, a payment processor, a treasury desk — wears these gaps and cannot see them until they have already cost something.

The failure sharpens under an autonomous agent on the treasury. An agent that rebalances or settles on a schedule will, unless something stops it, submit against whatever reserve figure it last read, or worse against a figure a redemption has already invalidated. “We never settle past proven reserves” has to be an on-chain invariant the agent cannot argue its way around, not a policy in a runbook.

2 Why Arc

Arc is Circle’s own settlement chain, with USDC as native gas and value and first-class access to Circle’s settlement primitives (CCTP [4], Wallets, Gateway) [3]. That makes it the natural home

for a proof-of-reserves settlement rail: the reserve asset, the settled asset, and the gas asset are the same regulated dollar, so the invariant “circulating never exceeds attested reserves” is expressed in one unit end to end, with no bridge-risk seam in the middle of the guarantee. On a general-purpose chain this product is a bolt-on; on Arc it is native to how value already moves. Because balances track USDC’s native six decimals, no unit-conversion step sits between the proof and the gate.

3 Design

ArcSettle is two self-contained contracts and an operator agent. The invariant is:

$$\text{totalSupply} + \text{amount} \leq \text{effectiveReserves},$$

checked on every settlement, where

$$\text{effectiveReserves} = \begin{cases} 0 & \text{if the attestation is stale or future-dated} \\ \max(0, R - D) & \text{otherwise,} \end{cases}$$

R is the attested USDC reserve figure, and D is redemption debt: USDC redeemed that the current attestation has not yet accounted for.

3.1 The proof path: ReserveAttestationOracle

Reserve figures do not enter the oracle because a privileged account says so. They enter because someone presents an EIP-712 [2] signature from a registered attestor over (reserves, attestedAt, nonce). The contract recovers the signer on-chain and refuses: unknown signers; replays (a strictly incrementing nonce); malleable signatures (low- s check, cf. EIP-2098 [7]); wrong-length or bad- v signatures; and backdated or future-dated attestations. Submission is permissionless — the signature, not the caller, is the authority — so liveness never depends on one relayer’s key. The attestor set is the only trust root, and it is governed asymmetrically: *adding* an attestor (the direction that could prove reserves out of nothing) is a two-day timelocked, publicly announced change; *removing* one (which can only shrink what the oracle believes) is immediate, so a leaked key is revocable in the block it is discovered. The digest is bound to the chain id and the contract address, so an attestation signed for one deployment cannot be replayed against another.

3.2 The gate: ArcSettlement

ArcSettlement is a minimal, correct ERC-20 [8] settlement-claim token (six decimals, checked arithmetic, custom errors, events on every state change) whose `settle` function reverts with `InsufficientReserves`, `StaleAttestation`, or `FutureAttestation` as appropriate. Three properties are load-bearing:

1. **The operator cannot conjure headroom.** The oracle and the freshness bound change only through a two-step, two-day-timelocked proposal that is public the moment it is proposed, and `maxAttestationAge` is clamped to [1 min, 7 days], so “turn the freshness check off” is not a reachable state.
2. **Redemption does not silently reopen headroom.** Redeeming frees supply but the freed amount is booked as `pendingRedemptions` and subtracted from attested reserves until a *new* attestation (higher `attestationId`) lands that has observed the outflow. This is the defence against the redeem-move-collateral-resettle attack.

3. **Time is handled defensively.** A future-dated timestamp reverts instead of reading as eternally fresh, and freshness is computed as `block.timestamp - attestedAt` so no addition can overflow.

Every `IReserveOracle` getter is `view`, so the gate reads the oracle with `STATICCALL`; a malicious oracle that tries to re-enter `settle` while being read cannot change state, and the settlement fails closed.

3.3 The operator agent

The agent (`arcsettle/`) mirrors the on-chain gate off-chain and refuses a settlement that would exceed proven reserves rather than broadcasting a doomed transaction. Ordering is deliberate: an amount outside the `uint256` domain is rejected first; then the reserve pre-check; and only a settlement that will actually be submitted consumes a slot from an action limiter, so a burst of bad proposals cannot lock out a good one. Every decision is written to an audit trail. In live mode the pre-check is a real `eth_call` dry-run whose revert reason is the contract’s own custom error, not a local guess.

4 Implementation and evaluation

The contracts target solc 0.8.24 and are self-contained (no external Solidity library), so the security surface is auditable in two files. The agent has no third-party runtime dependency: Keccak-256 [1], selectors, and ABI word codecs are implemented in about a hundred lines and pinned to published test vectors, so “live mode” can never degrade into an import error.

We evaluate by test, using Foundry [6] for the contracts:

- **59 Solidity tests** (`forge test`): 36 on the settlement gate (the reserve invariant, freshness, redemption accounting, timelocked oracle rotation, re-entrancy, ERC-20 plumbing, and a supply-versus-reserves fuzz test) and 23 on the oracle (EIP-712 recovery, replay, malleability, backdating, and the timelocked attester set). Each defence has a test that goes red when the defence is removed.
- **75 Python tests** (`pytest`): the operator agent against an offline model of the contract, the dependency-free ABI layer pinned to solc’s own selectors, and live-mode configuration validation and revert decoding against a fake node. Many carry a named Solidity counterpart, so the two suites read side by side.

The redeem-then-resettle defence is the sharpest test: `test_RedeemThenResettleAgainstOldAttestationRev` settles 10^6 USDC against 10^6 reserves, redeems 4×10^5 , restates the same proof (no new id) as if the USDC had been moved out, and asserts the re-settle reverts, leaving supply at 6×10^5 .

5 Limitations

This is milestone 1, and the honest scope is narrow.

- **No mainnet**, and none planned under this grant. All work is Arc testnet only; the contract is not deployed anywhere yet (testnet deployment is milestone 2).

- **No live broadcast.** Live mode is read-only: reads and settle dry-runs are real `eth_calls`, but broadcasting a signed settlement needs a funded testnet key and a signer this repo does not ship. That path, with a Circle-primitive custody seam, is milestone 3.
- **No users, no revenue, no audit, no partnership with Circle.** The substitute for traction is mutation-tested code and a candid scope map, not usage numbers.

6 Conclusion

ArcSettle makes “an agent provably cannot settle past proven reserves” an on-chain invariant on Arc, denominated end to end in USDC, with a redemption defence that naive reserve gates lack and an operator agent that never burns gas on a settlement the chain would reject. Milestone 1 delivers the contract and the proof path, tested. The durable contribution is a standard, tested answer to how an agent settles in USDC on Arc without ever exceeding proven reserves, established before mainnet.

References

- [1] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak reference, version 3.0. <https://keccak.team/files/Keccak-reference-3.0.pdf>, 2011.
- [2] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. Ethereum Improvement Proposals, no. 712, 2017. <https://eips.ethereum.org/EIPS/eip-712>.
- [3] Circle. Arc: an open layer-1 blockchain purpose-built for stablecoin finance. <https://www.circle.com/arc>, 2026. Accessed 2026-08-25.
- [4] Circle. Cross-chain transfer protocol (cctp). <https://www.circle.com/cross-chain-transfer-protocol>, 2026. Accessed 2026-08-25.
- [5] Circle. USDC: a fully-reserved, dollar-backed stablecoin. <https://www.circle.com/usdc>, 2026. Accessed 2026-08-25.
- [6] Foundry contributors. Foundry: a portable and modular toolkit for Ethereum application development. <https://github.com/foundry-rs/foundry>, 2024.
- [7] Richard Moore and Nick Johnson. EIP-2098: Compact signature representation. Ethereum Improvement Proposals, no. 2098, 2019. <https://eips.ethereum.org/EIPS/eip-2098>.
- [8] Fabian Vogelsteller and Vitalik Buterin. EIP-20: Token standard. Ethereum Improvement Proposals, no. 20, 2015. <https://eips.ethereum.org/EIPS/eip-20>.